

NOTES: “AI Motivational Quote Agent” — Java on VS Code + Python (Multi-class, Reusable, with Error Handling)

Beginner-friendly, step-by-step guide to:

1. Create/OpenAI account & API key,
2. Set environment variables securely,
3. Build a **multi-class Java project** in VS Code (clean architecture, try/catch, logging),
4. Build a **multi-module Python variant** with the same pattern,
5. Run, verify output, and (optionally) schedule.

📖 Index

1. [What you'll build](#)
2. [Prerequisites](#)
3. [Create OpenAI account & API key](#)
4. [Set your API key as an environment variable](#)
5. [Java on VS Code \(full walkthrough\)](#)
 - 5.1 [Install JDK & VS Code extensions](#)
 - 5.2 [Create the project \(Gradle\)](#)
 - 5.3 [Add OpenAI SDK dependency](#)
 - 5.4 [Project structure](#)
 - 5.5 [Java code \(multi-class, reusable\)](#)
 - 5.6 [Run & debug in VS Code](#)
 - 5.7 [Package JAR \(optional\)](#)
6. [Python \(multi-module variant\)](#)
 - 6.1 [Virtual environment & install](#)
 - 6.2 [Python project structure](#)
 - 6.3 [Python code \(client, agent, app\)](#)
 - 6.4 [Run & verify](#)
7. [Scheduling \(optional\)](#)
8. [Troubleshooting & tips](#)
9. [Security checklist](#)

[Back to top](#)

1) What you'll build

A tiny “AI Motivational Quote Agent” that:

- Uses a **reusable client** class to talk to OpenAI (“ChatGPT”) API.
- Uses an **agent** class that crafts a focused prompt for **IT students preparing for MAANG/FAANG** interviews (short motivation + prep bullets).
- An **app/entry point** that calls the agent and **writes the reply to `gpt_output.txt`** (UTF-8).
- Implemented twice: **Java (VS Code)** and **Python**—both with clean structure and robust error handling.

[Back to top](#)

2) Prerequisites

- A computer with internet access and permission to install packages.
- An OpenAI account with billing enabled for API usage.
- **Java 17** (or 11/21) + **VS Code** (latest).
- **Python 3.8+** (if you'll also do the Python variant).

[Back to top](#)

3) Create OpenAI account & API key

1. Sign in at the OpenAI Platform.
2. Go to **API keys** → **Create new secret key**.
3. Copy it once. You **won't** see it again—store securely (password manager).

Never commit keys to Git. Use environment variables or a secret manager.

[Back to top](#)

4) Set your API key as an environment variable

Use the name `OPENAI_API_KEY`.

- **Windows (PowerShell)**

```
[System.Environment]::SetEnvironmentVariable("OPENAI_API_KEY", "sk-REPLACE_ME", "User")  
# Open a NEW terminal to inherit it:  
$env:OPENAI_API_KEY
```

- **macOS / Linux (bash/zsh)**

```
echo 'export OPENAI_API_KEY="sk-REPLACE_ME"' >> ~/.zshrc  
source ~/.zshrc  
echo "$OPENAI_API_KEY"
```

[Back to top](#)

5) Java on VS Code (full walkthrough)

5.1 Install JDK & VS Code extensions

- Install **JDK 17** (recommended). Verify:

```
java -version  
javac -version
```

- In VS Code, install:
 - **Extension Pack for Java** (by Microsoft) (includes Language Support for Java, Debugger, Test Runner, Maven, Project Manager)
 - *(Optional)* **Gradle for Java** (handy tasks panel)

5.2 Create the project (Gradle)

1. VS Code **Command Palette** → **"Java: Create Java Project"** → **Gradle** → **No template**.
2. Name it: `ai-quote-agent`.
3. Choose a location, then open the folder.

5.3 Add OpenAI SDK dependency

Open `build.gradle` and add:

```
dependencies {  
    implementation("com.openai:openai-java:3.5.2") // use latest available  
}
```

Save. VS Code/Gradle will resolve the dependency.

5.4 Project structure

```

ai-quote-agent/
  build.gradle
  settings.gradle
  gradlew / gradlew.bat
  src/
    main/
      java/
        com/
          algorithms365/
            agent/
              App.java
              OpenAIChatClient.java
              AIMotivationalQuoteAgent.java

```

If the `com/algorithms365/agent` path doesn't exist, create it.

5.5 Java code (multi-class, reusable)

Design:

- `OpenAIChatClient` : Thin wrapper around SDK (timeouts, single method to complete).
- `AIMotivationalQuoteAgent` : Builds persona & output format for MAANG/FAANG prep.
- `App` : Entry point, argument parsing, file writing, try/catch, logging.

`OpenAIChatClient.java`

```

package com.algorithms365.agent;

import com.openai.client.OpenAIClient;
import com.openai.client.okhttp.OpenAIOkHttpClient;
import com.openai.models.ChatModel;
import com.openai.models.chat.completions.ChatCompletion;
import com.openai.models.chat.completions.ChatCompletionCreateParams;

import java.time.Duration;
import java.util.Objects;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 * Thin wrapper around the OpenAI Java SDK for chat completions.
 * - Centralizes client creation and configuration (timeouts).
 * - Single method to request a completion and return text.
 */
public class OpenAIChatClient {
    private static final Logger LOG = Logger.getLogger(OpenAIChatClient.class.getName());
    private final OpenAIClient client;
    private final ChatModel defaultModel;

    public OpenAIChatClient(ChatModel defaultModel) {
        this.client = OpenAIOkHttpClient.fromEnv(builder -> builder
            .callTimeout(Duration.ofSeconds(120))
            .connectTimeout(Duration.ofSeconds(30))
            .readTimeout(Duration.ofSeconds(120))
            .writeTimeout(Duration.ofSeconds(120))
        );
        this.defaultModel = Objects.requireNonNull(defaultModel, "defaultModel must not be null");
    }

    /**
     * Send a user prompt and return the first choice text.
     * @param prompt user text
     * @param maxTokens maximum tokens (nullable => default 200)
     * @return trimmed response text
     * @throws RuntimeException on network/auth/model errors

```

```

*/
public String complete(String prompt, Long maxTokens) {
    try {
        ChatCompletionCreateParams params = ChatCompletionCreateParams.builder()
            .addUserMessage(prompt)
            .model(defaultModel)
            .maxTokens(maxTokens != null ? maxTokens : 200L)
            .build();

        ChatCompletion completion = client.chat().completions().create(params);

        if (completion.choices() == null || completion.choices().isEmpty()) {
            throw new RuntimeException("OpenAI response had no choices.");
        }
        String text = completion.choices().get(0).message().content();
        if (text == null) {
            throw new RuntimeException("OpenAI response content was null.");
        }
        return text.trim();
    } catch (Exception ex) {
        LOG.log(Level.SEVERE, "OpenAI chat completion failed: " + ex.getMessage(), ex);
        throw new RuntimeException(
            "Failed to get a response from OpenAI. " +
            "Check OPENAI_API_KEY, network, and model access.", ex);
    }
}
}

```

AIMotivationalQuoteAgent.java

```

package com.algorithms365.agent;

import java.util.Objects;

/**
 * Agent that generates a short motivational + preparation snippet
 * for IT students targeting MAANG/FAANG interviews.
 */
public class AIMotivationalQuoteAgent {

    private final OpenAIChatClient chatClient;

    public AIMotivationalQuoteAgent(OpenAIChatClient chatClient) {
        this.chatClient = Objects.requireNonNull(chatClient);
    }

    /**
     * Build a focused prompt with persona and constraints, then call OpenAI.
     * @param studentName optional personalization
     * @param focusArea e.g., "DSA and System Design"
     */
    public String generateAdvice(String studentName, String focusArea) {
        String name = (studentName == null || studentName.isBlank()) ? "student" : studentName.trim();
        String area = (focusArea == null || focusArea.isBlank()) ? "algorithms, data structures, and system design" : focusArea.trim();

        String prompt = """
            You are the "AI Motivational Quote Agent" for IT students targeting MAANG/FAANG roles.
            Persona:
            - Mentor-like, concise, specific, and encouraging.
            - Output two parts: (1) Motivation (2-3 lines), (2) Preparation Snippet (3-5 bullets).
            - Concrete advice; avoid fluff. Keep under ~120 words.

            Context:
            - Student name: %s
            - Focus area: %s
            - Companies: MAANG/FAANG (Meta, Apple, Amazon, Netflix, Google; plus similar tier orgs).
            - Emphasize disciplined practice, patterns, interview hygiene.

            Output format:
            Motivation:
            • <short sentence>
            • <short sentence>

            Preparation:
            • <bullet 1>
            • <bullet 2>
            • <bullet 3>
            • <optional bullet 4-5>
            """.formatted(name, area);

        return chatClient.complete(prompt, 180L);
    }
}

```

App.java

```

package com.algorithms365.agent;

import com.openai.models.ChatModel;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.time.LocalDateTime;

```

```

import java.util.Optional;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 * CLI entry point:
 * - Optional args: studentName, focusArea
 * - Invokes agent and writes output to gpt_output.txt (UTF-8)
 * - Robust try/catch and logging
 */
public class App {
    private static final Logger LOG = Logger.getLogger(App.class.getName());

    public static void main(String[] args) {
        String studentName = (args.length >= 1 && !args[0].isBlank()) ? args[0] : "student";
        String focusArea = (args.length >= 2 && !args[1].isBlank()) ? args[1] : "DSA and System Design";

        // Guard: API key must exist
        if (System.getenv("OPENAI_API_KEY") == null) {
            LOG.severe("OPENAI_API_KEY is not set.");
            System.err.println("ERROR: Set OPENAI_API_KEY in your environment before running.");
            System.exit(1);
        }

        OpenAIChatClient client = new OpenAIChatClient(ChatModel.GPT_4_1);
        AIMotivationalQuoteAgent agent = new AIMotivationalQuoteAgent(client);

        try {
            String advice = agent.generateAdvice(studentName, focusArea);
            Path out = Paths.get("gpt_output.txt");
            writeUtf8(out, advice);
            LOG.info(() -> "Advice generated and written to: " + out.toAbsolutePath());
            System.out.println("SUCCESS: Wrote advice to " + out.toAbsolutePath());
        } catch (RuntimeException ex) { // OpenAI / runtime errors
            LOG.log(Level.SEVERE, "AI generation failed: " + ex.getMessage(), ex);
            System.err.println("ERROR: " + ex.getMessage());
            System.exit(2);
        } catch (Exception ex) { // last-resort
            LOG.log(Level.SEVERE, "Unexpected error: " + ex.getMessage(), ex);
            System.err.println("FATAL: " + ex.getMessage());
            System.exit(3);
        }
    }

    private static void writeUtf8(Path path, String content) throws IOException {
        String header = """
            =====
            AI Motivational Quote Agent
            Timestamp: %s
            =====

            """.formatted(LocalDateTime.now());

        try {
            Files.writeString(path, header + content + System.lineSeparator(), StandardCharsets.UTF_8,
                StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE);
        } catch (NoSuchFileException nsf) {
            if (path.getParent() != null) {
                Files.createDirectories(path.getParent());
                Files.writeString(path, header + content + System.lineSeparator(), StandardCharsets.UTF_8,
                    StandardOpenOption.CREATE, StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE);
            } else {
                throw nsf;
            }
        }
    }
}

```

```
}  
    }  
}  
}
```

5.6 Run & debug in VS Code

- **Terminal (Gradle):**

```
./gradlew run  
# with args  
./gradlew run --args='Mahesh "Operating Systems & DSA"'
```

- **Debug:** Open `App.java` → click the **Run** (▶) next to `main` to launch the debugger. To pass args, create a `launch.json` and set `"args": ["Mahesh", "System Design"]`.
- After a successful run, open `gpt_output.txt` at project root.

5.7 Package JAR (optional)

- **Fat/uber JAR (Gradle Shadow):** Add to `build.gradle`:

```
plugins {  
    id "com.github.johnrengelman.shadow" version "8.1.1"  
}
```

Then:

```
./gradlew shadowJar  
java -jar build/libs/ai-quote-agent-all.jar
```

[Back to top](#)

6) Python (multi-module variant)

6.1 Virtual environment & install

```
python -m venv .venv  
# Windows  
.venv\Scripts\activate  
# macOS/Linux  
source .venv/bin/activate  
  
pip install --upgrade openai
```

6.2 Python project structure

```
ai_quote_agent_py/  
  app.py  
  ai_motivational_agent.py  
  openai_client.py  
  requirements.txt
```

(Optional) Put your code into a package folder if you prefer (`src/` layout, etc.).

6.3 Python code (client, agent, app)

`openai_client.py`

```

"""
Reusable OpenAI chat client wrapper.
Reads OPENAI_API_KEY from environment (do NOT hard-code).
"""

from openai import OpenAI

class OpenAIChatClient:
    def __init__(self, model: str = "gpt-4o-mini"):
        # auto-reads OPENAI_API_KEY from environment
        self._client = OpenAI()
        self._model = model

    def complete(self, prompt: str, max_tokens: int | None = 200) -> str:
        """
        Send a user prompt to Chat Completions and return text.
        Raises RuntimeError on failure.
        """
        try:
            resp = self._client.chat.completions.create(
                model=self._model,
                messages=[{"role": "user", "content": prompt}],
                max_tokens=max_tokens or 200,
            )
            choices = getattr(resp, "choices", None)
            if not choices:
                raise RuntimeError("OpenAI response had no choices.")
            text = choices[0].message.content
            if not text:
                raise RuntimeError("OpenAI response content was empty.")
            return text.strip()
        except Exception as ex:
            raise RuntimeError(
                "Failed to get response from OpenAI. "
                "Check OPENAI_API_KEY, network, and model access."
            ) from ex

```

ai_motivational_agent.py

```

"""
Agent that crafts a persona-driven prompt for MAANG/FAANG prep advice.
"""

from openai_client import OpenAIChatClient

class AIMotivationalQuoteAgent:
    def __init__(self, chat_client: OpenAIChatClient):
        self._chat = chat_client

    def generate_advice(self, student_name: str | None, focus_area: str | None) -> str:
        name = (student_name or "student").strip()
        area = (focus_area or "algorithms, data structures, and system design").strip()

        prompt = f"""
You are the "AI Motivational Quote Agent" for IT students targeting MAANG/FAANG roles.
Persona:
- Mentor-like, concise, specific, and encouraging.
- Output two parts: (1) Motivation (2-3 lines), (2) Preparation Snippet (3-5 bullets).
- Concrete advice; avoid fluff. Keep under ~120 words.

Context:
- Student name: {name}
- Focus area: {area}
- Companies: MAANG/FAANG (Meta, Apple, Amazon, Netflix, Google; similar tier firms).
- Emphasize disciplined practice, patterns, interview hygiene.

Output format:
Motivation:
• <short sentence>
• <short sentence>

Preparation:
• <bullet 1>
• <bullet 2>
• <bullet 3>
• <optional bullet 4-5>
"""

        return self._chat.complete(prompt, max_tokens=180)

```

app.py

```

"""
CLI entry for AI Motivational Quote Agent (Python).
- Optional args: student name, focus area
- Writes UTF-8 text to gpt_output.txt
- Robust error messages for beginners
"""

import os
import sys
from pathlib import Path
from datetime import datetime

from openai_client import OpenAIChatClient
from ai_motivational_agent import AIMotivationalQuoteAgent

HEADER = """\
=====
AI Motivational Quote Agent
Timestamp: {ts}
=====

"""

def main(argv: list[str]) -> int:
    if not os.getenv("OPENAI_API_KEY"):
        print("ERROR: OPENAI_API_KEY is not set in your environment.", file=sys.stderr)
        return 1

    student_name = argv[1] if len(argv) >= 2 else "student"
    focus_area = argv[2] if len(argv) >= 3 else "DSA and System Design"

    try:
        client = OpenAIChatClient(model="gpt-4o-mini")
        agent = AIMotivationalQuoteAgent(client)
        advice = agent.generate_advice(student_name, focus_area)

        out_path = Path("gpt_output.txt")
        out_path.write_text(HEADER.format(ts=datetime.now()) + advice + "\n", encoding="utf-8")
        print(f"SUCCESS: Wrote advice to {out_path.resolve()}")
        return 0

    except Exception as ex:
        print(f"ERROR: {ex}", file=sys.stderr)
        return 2

if __name__ == "__main__":
    sys.exit(main(sys.argv))

```

requirements.txt

```
openai>=1.40.0
```

6.4 Run & verify

```

# from ai_quote_agent_py/ (venv active)
python app.py
# with args
python app.py Mahesh "Operating Systems & DSA"

```

Check **gpt_output.txt** at the project root.

[Back to top](#)

7) Scheduling (optional)

- **Windows Task Scheduler**
 - Program/script: `java -jar C:\path\ai-quote-agent-all.jar` (Java) or `python C:\path\ai_quote_agent_py\app.py` (Python)
 - Trigger: Daily at **09:00** (Asia/Kolkata).
- **Linux/macOS (cron)**

```
# Daily at 09:00 IST (adjust path/interpreter)
0 9 * * * /usr/bin/env bash -lc 'cd /path/to/project && java -jar ai-quote-agent-all.jar >> run.log 2>&1'
# or Python:
0 9 * * * /usr/bin/env bash -lc 'cd /path/to/ai_quote_agent_py && /usr/bin/python3 app.py >> run.log 2>&1'
```

Ensure the environment (incl. `OPENAI_API_KEY`) is available to the scheduled process.

[Back to top](#)

8) Troubleshooting & tips

- **401 Unauthorized** → Missing/invalid `OPENAI_API_KEY`. Re-open terminal after setting.
- **404 model_not_found** → Switch to a chat-capable model your account can access.
- **Network/Proxy** → Configure system/JVM proxy (Java) or `HTTPS_PROXY` (Python).
- **Timeouts** → We set generous timeouts in Java; retry later if service is busy.
- **Encoding** → We always write **UTF-8**; open the file with UTF-8 aware editors.

[Back to top](#)

9) Security checklist

- ☒ **Never** hard-code API keys; use env vars or a secret manager.
- ☒ Don't print keys in logs or error messages.
- ☒ Add `gpt_output.txt` and any transient files to `.gitignore` if needed.
- ☒ For teams: use per-developer keys and least-privilege project scopes.